I have the following code:

```
export function getProviderLogoPathForDataSo
  ds: DataSourceType
): string | null {
  const provider = ds.connectorProvider;

  if (!provider) {
    return null;
  }

  switch (provider) {
    case "notion":
      return `/static/notion_32x32.png`;

    case "slack":
      return `/static/slack_32x32.png`;

    case "github":
      return `/static/github_black_32x32.png

    default:
      // eslint-disable-next-line @typescrip
      ((_provider: never) => {
        // cannot happen
        // this is to make sure we handle a
      })(provider);
      return null;
  }
}
```

I use this `never` case to make sure all enum values are handled. Is there a more idiomatic way to do this ?

The way you are handling this is generally correct and idiomatic. The `never` type is used to ensure that all cases are handled and that new cases aren't forgotten. The technique is called Exhaustiveness checking in TypeScript.

However, instead of using an inline arrow function, you could use a helper function to assert exhaustiveness. This would improve readability and remove the necessity for eslint disabling:

typescript Copy code

```typescript
export function getProviderLogoPathForDataSource(
  ds: DataSourceType
): string | null {
  const provider = ds.connectorProvider;

  if (!provider) {
    return null;
  }

  switch (provider) {
    case "notion":
      return `/static/notion_32x32.png`;

    case "slack":
      return `/static/slack_32x32.png`;

    case "github":
      return `/static/github_black_32x32.png`;

    default:
      return assertNever(provider);
  }
}

function assertNever(value: never): never {
    throw new Error(`Unexpected value '${value}'`);
}
```

In the `assertNever` function, we are throwing an error if an unexpected value is encountered. This could be useful during development because it would crash your

application at the source of the problem if you forget to add a case when extending your `provider`. Of course, the thrown error would be a problem in a production application, so you need to make sure that your switch is exhaustive during testing and before deployment.